

SeniorLink

Documento técnico para a apresentação

Análise do código disponível em 29 de setembro de 2026

Material de apoio para explicar como o projeto foi desenvolvido, quais recursos existem e como as integrações futuras poderiam funcionar. A análise contempla o aplicativo Android e o site de exposição, que são projetos separados.

A proposta do projeto

O SeniorLink propõe apoiar a segurança e a autonomia da pessoa idosa por meio de localização compartilhada, informações úteis em emergências, lembretes, contatos de confiança e inclusão digital. O protótipo atual permite apresentar essa proposta e percorrer parte das telas.

O que a equipe pode afirmar hoje

- Existe um aplicativo Android nativo escrito em Kotlin, com telas em XML e Jetpack Compose, preparado para execução pelo Android Studio.
- A navegação, os campos do perfil, a lista temporária de lembretes, os tutoriais e a confirmação demonstrativa de emergência aparecem no código.
- Há também um site independente em HTML, CSS e JavaScript para exibição no notebook, com recursos locais e simulações de uso.
- Localização real, compartilhamento entre aparelhos, autenticação, alertas e integração com uma tag física são etapas futuras. Não há uma API externa desses serviços conectada nesta versão.

Uma fala técnica de aproximadamente um minuto

“O SeniorLink é um protótipo de aplicativo Android pensado para apoiar a pessoa idosa sem tirar sua autonomia. O aplicativo foi desenvolvido em Kotlin e combina telas XML com Jetpack Compose. Hoje já conseguimos apresentar a navegação, os campos de perfil, lembretes temporários e orientações de uso do celular. Também temos um site para demonstrar a experiência no evento. A próxima etapa técnica é conectar a localização do aparelho, exibir essa informação no mapa e permitir o acesso de um responsável autorizado. Para isso, estamos estudando serviços de localização, mapas e sincronização de dados. Essas integrações ainda não estão prontas no protótipo.”

As afirmações sobre o código estão detalhadas nas páginas 2 a 4. As páginas 5 e 6 apresentam possibilidades técnicas, e a página 7 reúne respostas para a banca. A base da análise está registrada na página 8.

1 Tecnologias do aplicativo

O aplicativo usa recursos nativos do Android. Os arquivos de aplicação examinados são Kotlin, mesmo estando dentro de uma pasta chamada java. Não foram encontrados arquivos de código-fonte .java nessa aplicação. (A1, A2)

Tecnologia ou ferramenta	Uso identificado e versão declarada
Kotlin	Linguagem dos 13 arquivos .kt da aplicação. O plugin de Compose declara 2.2.10 no catálogo.
Jetpack Compose	Interface da área principal. Compose BOM 2026.02.01; UI, Graphics, Preview e Material 3 geridos pelo BOM.
XML e Android Views	Layouts da abertura e das três telas de apresentação inicial.
AppCompat e ConstraintLayout	Apoio às telas XML. AppCompat 1.6.1 e ConstraintLayout 2.1.4.
Activity e Core KTX	Integração de telas e utilitários Android. Activity Compose 1.8.0; Activity KTX 1.13.0; Core KTX 1.10.1.
Lifecycle e Material	Dependências declaradas: Lifecycle Runtime KTX 2.6.1 e Material Components 1.10.0.
Gradle e plugin Android	Automação da compilação. Gradle Wrapper 9.5.0 e Android Gradle Plugin 9.3.3.
Android SDK	compileSdk e targetSdk 37; minSdk 24, correspondente ao Android 7.0.
Java e JDK	Compatibilidade Java 11 nas opções de compilação. A configuração do daemon Gradle solicita JDK 25.
Android Studio e ADB	Ambiente de desenvolvimento e ferramenta de comunicação com o aparelho para instalar e abrir o APK.
Ferramentas de teste	JUnit 4.13.2, AndroidX Test JUnit 1.1.5, Espresso 3.5.1 e bibliotecas de teste Compose.
Git e GitHub	Controle de versões e repositórios. O repositório local do Android contém as branches main e develop.

Versões declaradas: os números acima vêm dos arquivos do projeto. O BOM e a resolução de dependências do Gradle podem selecionar outras versões transitivas. O número 2.2.10 identifica o plugin Compose declarado; não é uma auditoria de todos os compiladores e artefatos resolvidos.

Resposta sobre Java: “O nosso código de aplicação está em Kotlin. Java aparece no ecossistema e nas configurações de compilação; isso não significa que desenvolvemos parte das telas em Java.”

2 Funcionamento atual do aplicativo

A abertura passa por SplashActivity e pelas três Activities de onboarding. Essas telas usam XML e Intent para avançar ou pular. A MainActivity abre a área em Compose. (A2)

Dentro dessa área, uma variável chamada telaAtual seleciona a tela com uma expressão when. Os dados dos formulários e lembretes usam remember e mutableStateOf: ficam na memória da interface. Não há camada de banco de dados, serviço remoto ou arquitetura MVVM estruturada identificada nessa versão.

Recurso	Comportamento encontrado no código
Abertura e início	Telas de apresentação, opção de pular e atalhos para as funções do aplicativo.
Perfil	Campos editáveis de identificação, saúde e contato. O botão de salvar ainda não tem ação; os dados não são persistidos.
Localização	Tela de proposta. Os botões ainda não consultam GPS, não exibem um mapa conectado e não compartilham coordenadas.
Lembretes	Permite adicionar e excluir itens de uma lista temporária. Ao sair da tela, a lista é perdida. Não agenda alarmes ou notificações.
Aprender	Quatro orientações textuais expansíveis: tamanho das letras, ligação, Wi-Fi e mensagens.
Emergência	Abre uma confirmação informando que o protótipo não envia alertas nem faz chamadas. Contato de confiança ainda não é configurado.
Login e cadastro	Não foram encontrados fluxos de autenticação implementados. As telas conceituais do banner não comprovam essa função no app.

APIs já presentes

API é a interface pela qual um programa utiliza funções de outro componente. O aplicativo já usa APIs locais do Android e de bibliotecas, como Intent, Activity, tratamento de barras do sistema e funções de Compose. Isso é diferente de ter uma API de localização ou um servidor externo integrado.

A leitura dos fontes, das dependências e do manifesto não encontrou integração com Firebase, Google Maps, cliente de localização, Bluetooth, cliente HTTP ou banco persistente. O manifesto principal também não declara permissões de localização. (A1, A3)

3 Site de exposição e desenvolvimento

Como foi feita a versão web

O site é um projeto independente do APK. Usa HTML para a estrutura, CSS para o visual e JavaScript para as interações. As ilustrações, imagens e vídeos usados na exibição estão armazenados na pasta do projeto. O package.json não declara dependências de execução ou um framework web. (W1)

A página inicial alterna dez composições baseadas nas telas Android ou as 16 telas do banner. Tem reprodução automática, pausa, setas e modo de exposição. As composições web são adaptações para apresentação, e não capturas de uma execução real do aplicativo. (W2)

A página Explorar o app contém um simulador com perfil fictício, localização fixa, lembretes, tutoriais e SOS demonstrativo. Não consulta GPS nem envia mensagens ou realiza chamadas. Os lembretes podem ser guardados no localStorage do navegador, quando ele está disponível; esse armazenamento é diferente da lista temporária do aplicativo Android. (W3)

APIs do navegador e uso sem internet

O site utiliza o DOM para atualizar a página, localStorage para os lembretes do simulador, Fullscreen API para tela cheia e Screen Wake Lock API quando o navegador a oferece. São recursos do navegador, não serviços remotos de rastreamento.

Para a exibição local, o index.html pode ser aberto no navegador com os arquivos da pasta preservados. Há também um servidor opcional em Node.js 18 ou superior, usando módulos nativos e a porta local 4173. Ele entrega os arquivos do site; não é um backend de contas, localização ou saúde. Recursos como tela cheia, tela ativa e armazenamento dependem do navegador.

GitHub Pages hospeda a versão pública estática. Ter uma cópia local permite a apresentação sem internet; isso não acrescenta rastreamento real ou comunicação entre celulares ao protótipo. (W1, W4)

Referências visuais e apoio de IA

O registro de créditos do site identifica o template Navigator como referência da versão inicial e os recursos visuais do repositório da equipe como base da identidade atual. Os créditos e a licença do material de terceiros estão em THIRD_PARTY_NOTICES.md. (W5)

O histórico de prompts e respostas do Claude não foi fornecido para esta análise. Portanto, não é possível listar com precisão “tudo que o Claude fez” ou atribuir cada arquivo a essa ferramenta. Este documento verifica o resultado presente nos projetos; o grupo deve relatar o uso de IA conforme o seu histórico real.

4 Proposta de integração de APIs

As opções abaixo formam uma sugestão de evolução técnica. Nenhuma delas foi integrada ao projeto nesta análise, e a escolha definitiva de fornecedores ainda depende da equipe.

Obter a posição do aparelho

O Fused Location Provider, do Google Play services, é uma opção para receber posições e atualizações de localização no Android. O aplicativo precisaria solicitar a permissão adequada, tratar recusas, verificar a precisão e a data da leitura e encerrar atualizações quando não forem necessárias. Uma posição antiga não deve aparecer como atual.

Referências: [FusedLocationProviderClient](#) · [Permissões de localização](#)

Mostrar a posição no mapa

O Maps SDK for Android pode exibir um mapa com marcador na latitude e longitude recebidas. O marcador é a representação visual de uma posição; ele não descobre sozinho onde a pessoa está. A configuração do serviço exige projeto Google Cloud, habilitação do SDK, chave de API e os requisitos de conta e faturamento do provedor.

Referências: [Marcadores no Maps SDK](#) · [Configuração do Maps SDK](#)

Compartilhar com o responsável

Uma possibilidade é combinar Firebase Authentication, para identificar os usuários, com Firebase Realtime Database, para armazenar e sincronizar a última posição. As regras de acesso precisariam permitir que somente a pessoa idosa e os responsáveis vinculados consultassem esses dados. Estar autenticado, por si só, não deve dar acesso à localização de outras pessoas.

Referências: [Leitura e gravação no Android](#) · [Regras de acesso e autenticação](#)

Fluxo proposto para dois celulares

1. A pessoa idosa autoriza a localização e escolhe com quem compartilhar.
2. Seu aparelho obtém uma posição com horário e precisão.
3. O aplicativo envia os dados ao serviço com controle de acesso.
4. O celular do responsável autorizado recebe a atualização.
5. O mapa mostra a posição e informa quando ela foi atualizada.

Esse fluxo é uma proposta de arquitetura. A atualização remota precisa de conectividade para chegar ao outro aparelho. Sem comunicação, a tela deve identificar a última informação recebida. A primeira validação pode usar o app aberto; funcionamento em segundo plano exige uma etapa própria de permissões, consumo de bateria e restrições do Android.

5 Tag e próximos passos

Se tag significa um marcador no mapa

Nesse caso, a tag pode ser um ícone associado ao identificador do usuário, colocado sobre o mapa na última posição recebida. Não é necessário inventar uma API separada de tag: o SDK de mapas já oferece marcadores e associação de dados. A origem das coordenadas e o controle de acesso continuam sendo responsabilidades de outros componentes.

Referências: [Marcadores e dados associados](#)

Se tag significa um dispositivo físico

A solução depende do hardware escolhido. Uma tag Bluetooth Low Energy pode comunicar dados a um celular ou receptor próximo. Bluetooth, sozinho, não transforma a tag em um rastreador GPS com internet própria. O Android oferece APIs para localizar dispositivos BLE e trocar dados, conforme os serviços e protocolos que o dispositivo disponibiliza.

Para acompanhar alguém à distância, seria necessário definir como a posição será obtida e transmitida: por um aparelho próximo, por infraestrutura compatível ou por um dispositivo com recursos próprios. Isso depende de alcance, bateria, rede e documentação do fabricante. Nenhum modelo de tag, protocolo ou SDK de fabricante foi identificado nos projetos analisados.

Referências: [Visão geral de Bluetooth Low Energy no Android](#)

Ordem sugerida de evolução

- Consolidar os dados locais: salvar perfil, contatos e lembretes de forma persistente, com tratamento de campos e recuperação ao reabrir o aplicativo.
- Validar localização com o app aberto: solicitar permissão e apresentar posição, precisão, horário e estados de erro.
- Adicionar contas e vínculo de confiança: definir quem pode consultar cada usuário e como o compartilhamento pode ser encerrado.
- Sincronizar dois aparelhos: enviar e receber a última posição, testar perda de rede e distinguir informação recente de informação desatualizada.
- Implementar e testar lembretes e pedidos de ajuda: definir agendamento, destinatários e comportamento de falha. Uma mensagem enviada não comprova que houve atendimento.
- Avaliar segundo plano e tag física depois da definição do hardware e das necessidades reais de uso.

Critérios para considerar a evolução validada

Verificar permissões negadas, aplicativo fechado, falta de rede, dados antigos e restrição de acesso a outro usuário. Avaliar também leitura das telas, tamanho dos controles e compreensão das ações com o público do projeto. O código atual não demonstra detecção automática de quedas nem resultados medidos de redução de acidentes.

6 Respostas para a apresentação

Como vocês desenvolveram o aplicativo?

“O aplicativo é nativo Android, escrito em Kotlin. A abertura usa layouts XML e a área principal usa Jetpack Compose. O Gradle organiza a compilação e as bibliotecas do projeto.”

Vocês usaram Java também?

“O código de aplicação que analisamos está em Kotlin. Há configurações de compatibilidade Java e uso do JDK nas ferramentas, mas não encontramos arquivos de aplicação escritos em Java.”

Qual API de localização já está funcionando?

“Ainda não temos essa integração. A tela atual apresenta a proposta. Uma opção de evolução é obter a posição pelo Fused Location Provider, exibi-la no Maps SDK e sincronizá-la com o responsável autorizado.”

Onde os dados ficam salvos?

“No Android, os campos e lembretes atuais ficam temporariamente na memória da tela. Não há banco persistente conectado. No simulador web, apenas os lembretes podem ficar no armazenamento local do navegador.”

O botão SOS liga ou manda mensagem?

“Nesta versão ele demonstra o fluxo e informa que não há envio real. A integração futura precisa definir o destinatário e testar o que acontece quando não há rede ou quando a mensagem não é recebida.”

Como seria a tag para mapear a pessoa idosa?

“Se for um ícone no mapa, é um marcador visual. Se for um aparelho físico, precisamos escolher o hardware e estudar sua comunicação. Uma tag Bluetooth depende de um receptor compatível para transmitir dados.”

Funciona sem internet?

“As telas locais do protótipo e o site copiado no notebook podem ser apresentados sem internet. Compartilhar novas posições com um familiar distante exige um meio de comunicação entre os aparelhos.”

Qual é a diferença entre o site e o aplicativo?

“O site é uma apresentação e um simulador em HTML, CSS e JavaScript. O aplicativo é o projeto Android em Kotlin. Eles não estão sincronizados e podem demonstrar recursos em estágios diferentes.”

Foi desenvolvido com inteligência artificial?

A equipe deve explicar o uso real das ferramentas e o trabalho de revisão que realizou. Este levantamento não recebeu o histórico do Claude, portanto não confirma quais trechos foram gerados por ele. Não atribuir a uma IA recursos ou decisões sem registro.

O aplicativo já foi validado para uso real?

“Estamos na fase de protótipo. A preparação técnica para executar o aplicativo não equivale a validar rastreamento, emergências ou uso com idosos. Essas funções ainda precisam ser implementadas e testadas.”

7 Base da análise e limites

A documentação descreve as cópias locais consultadas em 29/09/2026. Os identificadores abaixo permitem conferir de qual versão as informações foram extraídas. As propostas de integração foram fundamentadas nas páginas oficiais vinculadas nas páginas 5 e 6, consultadas na mesma data.

Projeto Android

Cópia local: Documents/SeniorLink-Android. Commit de referência:

6314373c7599b4bbc4a4e5410a56de746f724fba. Pacote: com.seniorlink.app; versão declarada: 1.0, código 1.

Referências: [Repositório fornecido pela equipe](#) · [Cópia de trabalho de Wallacy](#)

A1 Configuração. app/build.gradle.kts; gradle/libs.versions.toml; gradle/wrapper/gradle-wrapper.properties; gradle/gradle-daemon-jvm.properties. Base para SDKs, dependências, ferramentas e versões declaradas.

A2 Interface e navegação. app/src/main/java/com/seniorlink/app/: MainActivity.kt, SplashActivity.kt e activity_onboarding2.kt, activity_onboarding3.kt, activity_onboarding4.kt; layouts em app/src/main/res/layout/. Base para o fluxo e a combinação XML/Compose.

A3 Funcionalidades. ui/screens/FeatureScreens.kt e HomeScreen.kt, dentro da pasta de código acima; app/src/main/AndroidManifest.xml. Base para estado temporário, botões demonstrativos e ausência de integrações de localização, alertas e autenticação.

A4 Alcance dos testes. ExampleUnitTest.kt testa a soma 2 + 2. ExampleInstrumentedTest.kt verifica o nome do pacote. Esses testes de exemplo não validam localização, alertas, persistência ou usabilidade. Nesta análise documental não foram executados builds, testes ou instalações.

Site de exposição

Cópia local: Documents/SeniorLink. Commit de referência: 2ce8d2dbc6f43ac30b36bceb65bebca94b904dda.

Referências: [Repositório do site](#) · [Endereço da apresentação](#)

W1 Tecnologias. index.html, explorar.html, css/ e package.json. Estrutura HTML/CSS/JavaScript e ausência de dependências de execução declaradas.

W2 Apresentação. showcase.js e app-screens.js. Sequência de cenas, composições visuais e modo de exposição.

W3 Simulador. index.js. Interações, exemplos fictícios, localStorage e ações simuladas de localização e emergência.

W4 Execução local. scripts/serve.mjs e README.md. Servidor de arquivos em 127.0.0.1, porta 4173, e orientação de abertura local.

W5 Créditos. THIRD_PARTY_NOTICES.md. Origem dos recursos visuais SeniorLink e referência ao template Navigator.